

WHY GREEDY?

Greedy algorithms make the **locally optimal choice** at each step, hoping that this choice leads to a globally optimal solution without ever looking back or backtracking!

THE GREEDY PROOF AHA

Greedy works **ONLY** when local choices never restrict optimal future choices (Greedy Choice Property). If choices depend on future outcomes, use **Dynamic Programming**!

3 CORE PATTERNS

- Farthest Reach (Jump Game):** Maintain `maxReach` boundary iteratively.
- Gas Station Circuit:** If total gas  $\geq$  total cost, a valid starting station is guaranteed!
- Kadane's Algorithm:** Reset running subarray sum to 0 if it drops below 0!

2 VISUALIZING JUMP GAME

Nums = [2, 3, 1, 1, 4]:

Index 0 (val 2)  $\rightarrow$  `maxReach = max(0, 0+2) = 2`  
Index 1 (val 3)  $\rightarrow$  `maxReach = max(2, 1+3) = 4!`  
`maxReach (4) >= Last Index (4)  $\rightarrow$  True!`

PREREQUISITES & COMPLEXITY REASONING

| Algorithm               | Time Complexity         | Auxiliary Space     |
|-------------------------|-------------------------|---------------------|
| Jump Game I & II        | <b>O(N)</b> single pass | <b>O(1)</b> space   |
| Gas Station (LC 134)    | <b>O(N)</b> single pass | <b>O(1)</b> space   |
| Kadane's Max Subarray   | <b>O(N)</b> single pass | <b>O(1)</b> space   |
| Task Scheduler (LC 621) | <b>O(N)</b> frequency   | <b>O(1)</b> hashmap |

CLUES THAT SCREAM GREEDY

- "Minimum jumps to reach end" / "Can reach last index"  $\rightarrow$  Farthest Reach
- "Complete circular gas circuit"  $\rightarrow$  Cumulative Tank Reset
- "Maximum sum contiguous subarray"  $\rightarrow$  Kadane's Algorithm

## 1 FARTHEST REACH TEMPLATE

```
public boolean canJump(int[] nums) {
    int maxReach = 0;
    for (int i = 0; i < nums.length; i++) {
        if (i > maxReach) return false; // Cannot reach index i!
        maxReach = Math.max(maxReach, i + nums[i]);
    }
    return true;
}
```

## 2 KADANE'S MAX SUBARRAY TEMPLATE

```
public int maxSubArray(int[] nums) {
    int maxSum = nums[0], currSum = 0;
    for (int num : nums) {
        currSum = Math.max(num, currSum + num);
        maxSum = Math.max(maxSum, currSum);
    }
    return maxSum;
}
```

PATTERN 1

JUMP GAME I (Farthest Reach Boundary) e.g. Jump Game (LC 55)

|                                                                                         |                                                                                                                                                                                                                                                                      |
|-----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>WHEN TO USE</b><br>Check if last index is reachable in $O(N)$ time and $O(1)$ space. | <b>TEMPLATE — LC 55</b> <pre>public boolean canJump(int[] nums) {     int maxReach = 0;     for (int i = 0; i &lt; nums.length; i++) {         if (i &gt; maxReach) return false;         maxReach = Math.max(maxReach, i + nums[i]);     }     return true; }</pre> |
|-----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

PATTERN 2

JUMP GAME II (Minimum Jumps BFS Window) e.g. Jump Game II (LC 45)

|                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>BFS WINDOW STRATEGY</b><br>Maintain current jump boundary `currEnd`. When loop reaches `currEnd`, increment `jumps++` and update `currEnd = maxReach`! | <b>TEMPLATE — LC 45</b> <pre>public int jump(int[] nums) {     int jumps = 0, currEnd = 0, maxReach = 0;     for (int i = 0; i &lt; nums.length - 1; i++) {         maxReach = Math.max(maxReach, i + nums[i]);         if (i == currEnd) {             jumps++;             currEnd = maxReach;         }     }     return jumps; }</pre> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

PATTERN 3 GAS STATION (Circular Tank Reset) e.g. Gas Station (LC 134)

**RESET CONDITION**  
If `totalGas >= totalCost`, a valid start exists! If `currTank < 0`, reset `start = i + 1` and `currTank = 0`.

TEMPLATE — LC 134

```
public int canCompleteCircuit(int[] gas, int[] cost) {
    int totalTank = 0, currTank = 0, start = 0;
    for (int i = 0; i < gas.length; i++) {
        int diff = gas[i] - cost[i];
        totalTank += diff; currTank += diff;
        if (currTank < 0) {
            start = i + 1; // Reset candidate station!
            currTank = 0;
        }
    }
    return totalTank >= 0 ? start : -1;
}
```

PATTERN 4 PARTITION LABELS (Last Index Boundary Tracking) e.g. Partition Labels (LC 763)

**LAST SEEN MAP**  
Record last occurrence index of each char. Loop string, update partition `end = Math.max(end, last[char])`. When `i == end`, partition complete!

TEMPLATE — LC 763

```
public List<Integer> partitionLabels(String s) {
    int[] last = new int[26];
    for (int i = 0; i < s.length(); i++) last[s.charAt(i) - 'a'] = i;
    List<Integer> res = new ArrayList<>();
    int start = 0, end = 0;
    for (int i = 0; i < s.length(); i++) {
        end = Math.max(end, last[s.charAt(i) - 'a']);
        if (i == end) {
            res.add(end - start + 1);
            start = i + 1;
        }
    }
    return res;
}
```

|                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>PATTERN 5TASK SCHEDULER (Max Frequency Idle Slot Formula)e.g. Task Scheduler (LC 621)</div>                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <div>FORMULA INTUITION</div> <p>Find max frequency <code>maxFreq</code>. Minimum slots needed <math>= (\text{maxFreq} - 1) \times (n + 1) + \text{countMaxFreq}</math>. Return <math>\max(\text{tasks.length}, \text{slots})</math>.</p> | <div>TEMPLATE — LC 621</div> <pre>public int leastInterval(char[] tasks, int n) {     int[] freq = new int[26];     for (char t : tasks) freq[t - 'A']++;     Arrays.sort(freq);     int maxFreq = freq[25];     int countMaxFreq = 0;     for (int f : freq) if (f == maxFreq) countMaxFreq++;     int partCount = maxFreq - 1;     int partLength = n - (countMaxFreq - 1);     int emptySlots = partCount * partLength;     int availableTasks = tasks.length - maxFreq * countMaxFreq;     int idles = Math.max(0, emptySlots - availableTasks);     return tasks.length + idles; }</pre> |
| <div>PATTERN 6VALID PARENTHESIS STRING (Min / Max Open Bounds)e.g. Valid Parenthesis String (LC 678)</div>                                                                                                                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <div>RANGE BOUND TRACKING</div> <p>Track <code>[cmin, cmax]</code> range of possible open brackets. <code>*</code> expands range <code>[cmin-1, cmax+1]</code>. Valid if <code>cmin == 0</code> at end!</p>                              | <div>TEMPLATE — LC 678</div> <pre>public boolean checkValidString(String s) {     int cmin = 0, cmax = 0;     for (char c : s.toCharArray()) {         if (c == '(') { cmin++; cmax++; }         else if (c == ')') { cmin--; cmax--; }         else { cmin--; cmax++; } // '*' can be ')', empty, or '('         if (cmax &lt; 0) return false;         cmin = Math.max(cmin, 0);     }     return cmin == 0; }</pre>                                                                                                                                                                        |

PATTERN 7

KADANE'S ALGORITHM (Maximum Subarray Sum) e.g. Maximum Subarray (LC 53)

RESET SUM MECHANICS

If `currSum < 0`, reset running sum to current number `num`. Update `maxSum = Math.max(maxSum, currSum)`.

TEMPLATE — LC 53

```
public int maxSubArray(int[] nums) {
    int maxSum = nums[0], currSum = 0;
    for (int num : nums) {
        currSum = Math.max(num, currSum + num);
        maxSum = Math.max(maxSum, currSum);
    }
    return maxSum;
}
```

GREEDY — WHICH ALGORITHM TO USE?

Start: What is the problem goal?

Reach end of array in jumps

→

**Farthest Reach / = BFS Window**  
``maxReach = Math.max(..., i + nums[i])``

Complete circular circuit

→

**Gas Station Reset**  
`Reset `start = i` if `tank < 0``

Max contiguous sum subarray

→

**Kadane's Algorithm**  
``currSum = Math.max(currSum, num)``

CPU Task Scheduling with Cooldown

→

**Max Freq Slot Formula**  
``partCount * partLength``

TRIGGER WORDS DIRECTORY

| Trigger Word           | Variant      | Action                                                                 |
|------------------------|--------------|------------------------------------------------------------------------|
| "Can reach last index" | Jump Game I  | Track <code>`maxReach = max(maxReach, i + nums[i])`</code>             |
| "Min jumps"            | Jump Game II | Update <code>`currEnd = maxReach`</code> on boundary                   |
| "Gas station circuit"  | Gas Station  | Reset <code>`start = i + 1`</code> when <code>`currTank &lt; 0`</code> |
| "Max subarray sum"     | Kadane's     | Reset <code>`currSum`</code> to <code>`num`</code> if negative         |

INTERVIEW TRADE-OFFS & FOLLOW-UPS

| Interviewer Asks                 | Your Answer                                                                                         |
|----------------------------------|-----------------------------------------------------------------------------------------------------|
| Greedy vs DP for Jump Game?      | Greedy runs in $O(N)$ time and $O(1)$ space. DP runs in $O(N^2)$ time! Greedy is optimal.           |
| Why does Gas Station reset work? | If sum from station $A$ to $B$ drops negative, NO station between $A$ and $B$ can reach $B$ either! |

EASY & MEDIUM — Jumps & Max Subarray

LC 53 · Maximum Subarray

EASY

**Pattern:** Kadane's Algorithm  
**Key Insight:** Reset sum if negative.

```
public int maxSubArray(int[] nums) {
    int mx = nums[0], cur = 0;
    for (int n : nums) {
        cur = Math.max(n, cur + n); mx = Math.max(mx, cur);
    }
    return mx;
}
```

LC 55 · Jump Game

MEDIUM

**Pattern:** Farthest Reach  
**Key Insight:** Return false if `i > maxReach`.

```
public boolean canJump(int[] nums) {
    int mx = 0;
    for (int i = 0; i < nums.length; i++) {
        if (i > mx) return false;
        mx = Math.max(mx, i + nums[i]);
    }
    return true;
}
```

LC 45 · Jump Game II

MEDIUM

**Pattern:** BFS Window Greedy  
**Key Insight:** `jumps++` when `i == currEnd`.

```
public int jump(int[] nums) {
    int j = 0, curEnd = 0, mx = 0;
    for (int i = 0; i < nums.length - 1; i++) {
        mx = Math.max(mx, i + nums[i]);
        if (i == curEnd) { j++; curEnd = mx; }
    }
    return j;
}
```

MEDIUM — Gas Station & Partition Labels

LC 134 · Gas Station

MEDIUM

**Pattern:** Tank Reset  
**Key Insight:** Reset `start = i + 1` if `currTank < 0`.

```
public int canCompleteCircuit(int[] gas, int[] cost) {
    int tot = 0, cur = 0, st = 0;
    for (int i = 0; i < gas.length; i++) {
        int d = gas[i] - cost[i]; tot += d; cur += d;
        if (cur < 0) { st = i + 1; cur = 0; }
    }
    return tot >= 0 ? st : -1;
}
```

LC 763 · Partition Labels

MEDIUM

**Pattern:** Last Index Window  
**Key Insight:** Partition when `i == end`.

```
public List<Integer> partitionLabels(String s) {
    int[] last = new int[26];
    for (int i = 0; i < s.length(); i++) last[s.charAt(i) - 'a'] = i;
    List<Integer> res = new ArrayList<>();
    int st = 0, end = 0;
    for (int i = 0; i < s.length(); i++) {
        end = Math.max(end, last[s.charAt(i) - 'a']);
        if (i == end) { res.add(end - st + 1); st = i + 1; }
    }
    return res;
}
```



MEDIUM — Task Scheduler & Parenthesis Bounds

LC 621 · Task Scheduler

MEDIUM

**Pattern:** Frequency Formula

**Key Insight:** Slots  $= (\text{maxFreq} - 1) \times (n + 1) + \text{countMax}$ .

```
public int leastInterval(char[] tasks, int n) {
    int[] f = new int[26]; for (char t : tasks) f[t - 'A']++;
    Arrays.sort(f); int maxF = f[25], countMax = 0;
    for (int v : f) if (v == maxF) countMax++;
    return Math.max(tasks.length, (maxF - 1) * (n + 1) + countMax);
}
```

MEDIUM — Min/Max Open Bounds

LC 678 · Valid Parenthesis String

MEDIUM

**Pattern:** Min/Max Open Bounds

**Key Insight:** Track `[cmin, cmax]`. Return `cmin == 0`.

```
public boolean checkValidString(String s) {
    int cmin = 0, cmax = 0;
    for (char c : s.toCharArray()) {
        if (c == '(') { cmin++; cmax++; }
        else if (c == ')') { cmin--; cmax--; }
        else { cmin--; cmax++; }
        if (cmax < 0) return false;
        cmin = Math.max(cmin, 0);
    }
    return cmin == 0;
}
```

| COMPLETE GREEDY PROBLEM LADDER SUMMARY |                          |                         |      |       |      |
|----------------------------------------|--------------------------|-------------------------|------|-------|------|
| LC #                                   | Problem                  | Pattern                 | Time | Space | Diff |
| 53                                     | Maximum Subarray         | Kadane's Algorithm      | O(N) | O(1)  | E    |
| 55                                     | Jump Game                | Farthest Reach Boundary | O(N) | O(1)  | M    |
| 45                                     | Jump Game II             | BFS Window Greedy       | O(N) | O(1)  | M    |
| 134                                    | Gas Station              | Circular Tank Reset     | O(N) | O(1)  | M    |
| 763                                    | Partition Labels         | Last Index Boundary     | O(N) | O(1)  | M    |
| 621                                    | Task Scheduler           | Max Frequency Formula   | O(N) | O(1)  | M    |
| 678                                    | Valid Parenthesis String | Min/Max Open Bounds     | O(N) | O(1)  | M    |

DRY RUN — Gas Station (gas=[1,2,3,4,5], cost=[3,4,5,1,2])

| Index | gas[i] - cost[i] | currTank | start Candidate | Action                                      |
|-------|------------------|----------|-----------------|---------------------------------------------|
| 0     | 1 - 3 = -2       | -2       | 1               | Reset `currTank = 0`, candidate `start = 1` |
| 1     | 2 - 4 = -2       | -2       | 2               | Reset `currTank = 0`, candidate `start = 2` |
| 2     | 3 - 5 = -2       | -2       | 3               | Reset `currTank = 0`, candidate `start = 3` |
| 3     | 4 - 1 = +3       | +3       | 3               | Valid tank (+3)                             |
| 4     | 5 - 2 = +3       | +6       | 3               | Return `start = 3` ✓                        |

MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Code Kadane's algorithm in  $O(1)$  space
- ☐ Write Jump Game I farthest reach in 2m
- ☐ Implement Jump Game II BFS window
- ☐ Code Gas Station circuit tank reset
- ☐ Write Partition Labels last occurrence map
- ☐ Code Task Scheduler max frequency formula
- ☐ Write Valid Parenthesis String min/max bounds
- ☐ Prove why Kadane's resets negative sums

MATHEMATICAL PROOF — KADANE'S ALGORITHM

**Claim:** Resetting running sum to 0 when it drops below 0 yields the maximum subarray sum.

**Proof:** Any prefix subarray with a negative sum  $\sum_{k=i}^j A[k] < 0$  can ONLY decrease the sum of any future subarray starting at  $j+1$ . Therefore, discarding this negative prefix is strictly optimal  $\rightarrow$   **$O(N)$  Time!**

GOLDEN RULES — NEVER FORGET

- Rule 1 — Verify Total Sum Before Returning**

In Gas Station, if total gas  $\geq$  total cost across the entire array, a valid starting station is guaranteed to exist!
- Rule 2 — Loop to N-1 for Jump Game II**

In Jump Game II, stop the loop at  $N - 2$  because once you reach  $N - 1$ , you are already at the destination and don't need another jump!